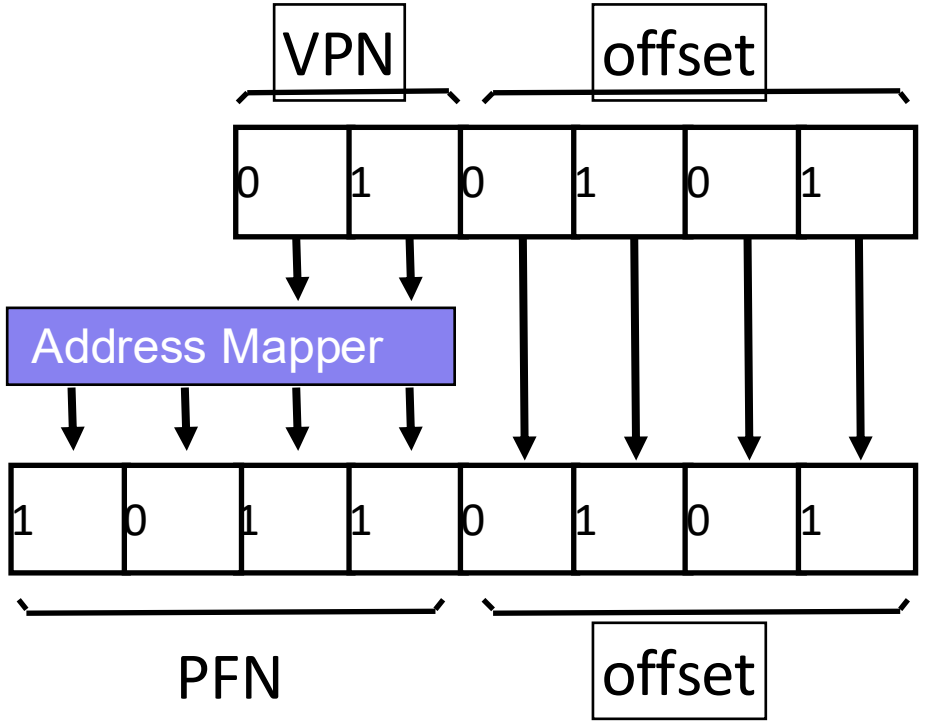
Memory Virtualization



Review: Virt ➔ Phys page mapping

Number of bits in the virtual address not equal the number of bits in the physical address



How should OS translate VPN to PFN?

For segmentation, OS used a formula (e.g., phys addr = virt_offset + base_reg)

For paging, OS needs a different mapping mechanism

Q: What data structure is good?

Linear Page Table

A simple option: an array

Review: Advantages of Paging

No external fragmentation

- Any page can be placed in any frame in physical memory

Fast to allocate and free at page granularity

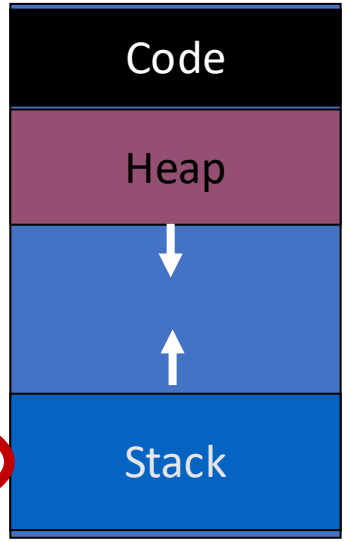
- Alloc: No searching for “fittable” free space. Any free page works
- Freeing is simple: No need to coalesce with adjacent free space
- Use simple data structs, e.g., bitmap to record the free/alloc frames

Can swap out portions of memory to disk! (remember time sharing?)

- Select a page size that matches disk block size
- Can still run a process when some pages are on disk
- Add a present bit to the page table entry

Review: Disadvantages of Paging

- Internal fragmentation: Page size may not match size needed by process
  - Wasted memory; wastage grows with larger pages
- Additional memory references → time-inefficient!
  - The page table must be stored in memory
  - MMU stores only base address of page table
- Storage for page tables substantial → space-inefficient!
  - Linear page table: Requires PTE for all pages in virt address space
    - Naively, page table entry needed even if a page is not allocated
  - Page tables must be allocated contiguously in physical memory
    - Due to linear access of page table entries, i.e, PTBR + VPN*sizeof(PTE)



Review: Avoid the simple linear page table

Use more efficient (but complex) data structures, instead of the simple big array

Any data structure is possible in principle!

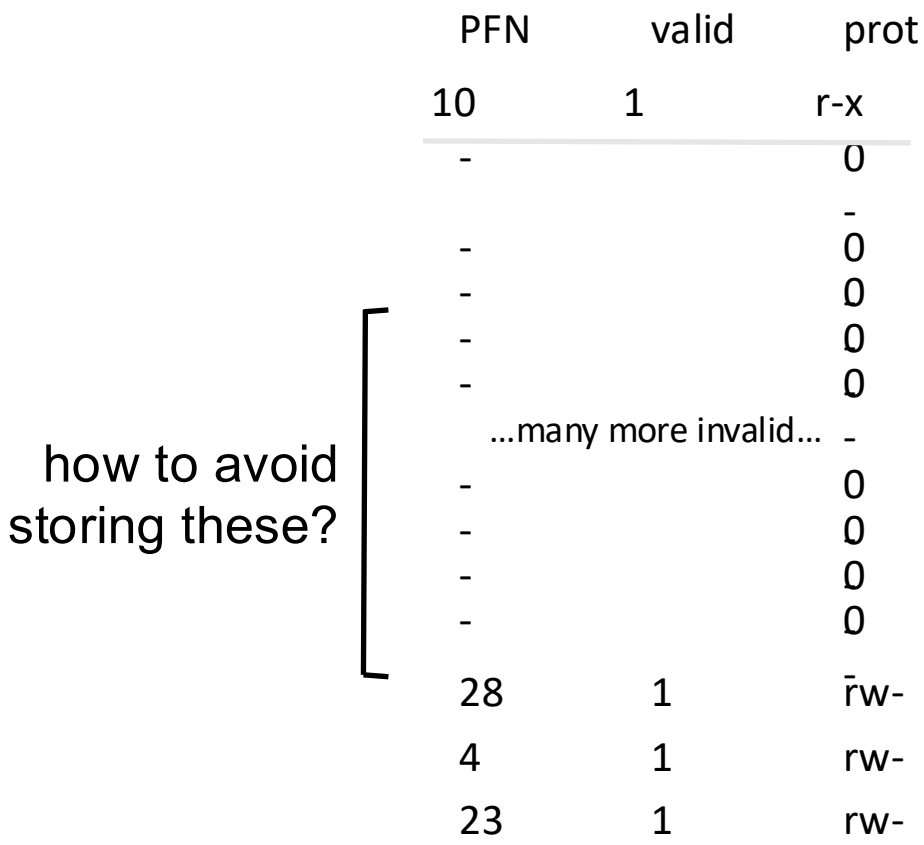
The hardware needs to know how to access it

… or a “cache” of it (we’ll see about TLBs later)

Some approaches

1. Inverted Pagetables
2. Segmented Pagetables
3. Multi-level Pagetables

Valid PTEs are Contiguous



Note “hole” in addr space: valids vs. invalids are clustered

How did OS avoid allocating holes in phys memory?

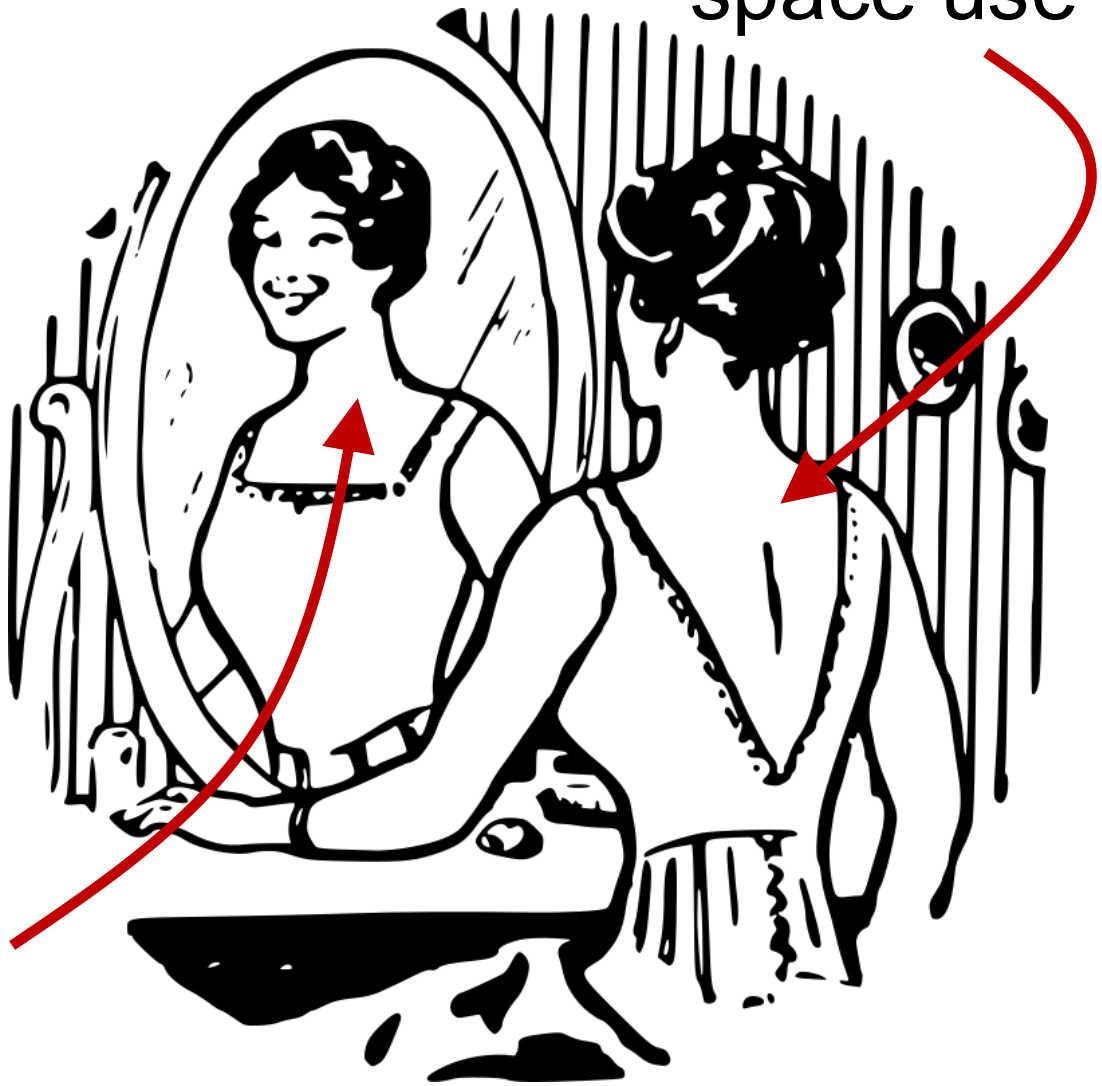
Valid PTEs are Contiguous

PFN	valid	prot
10	1	r-x
-		0
-		0
-		0
-		0
-		0
...many more invalid...		-
-		0
-		0
-		0
-		0
28	1	rw-
4	1	rw-
23	1	rw-

how to avoid storing these?

Page table entry space use

Virt address space use



Approach 2: Segmented Page Tables

Divide address space into segments (code, heap, stack)

- Segments can be variable length

Divide each segment into fixed-sized pages

Virtual address divided into three portions



Ideas

- Each segment has a page table
- Each segment tracks the base (physical address) and bounds of the **page table** for that segment

Combining Paging and Segmentation

seg # (4 bits)	page number (8 bits)	page offset (12 bits)

seg	base	bounds	R W
0	0x002000	0xff (255)	1 0
1	0x000000	0x00	0 0
2	0x001000	0x0f (15)	1 1

0x002070 read: 0x004070
0x202016 read: 0x003016
0x104c84 read: error
0x010424 write: error
0x210014 write: error
0x203568 read: 0x02a568

...
0x01f
0x011
0x003
0x02a
0x013
...
0x00c
0x007
0x004
0x00b
0x006
...

0x001000

0x002000

Linear Page table per segment!



Advantages of Segments

- Supports sparse address spaces
  - Decreases size of page tables
  - If segment not used, not needed for page table

Advantages of Pages

- No external fragmentation
- Segments can grow without any reshuffling
- Can afford to swap some pages to disk (we'll see this soon)

Advantages of Both

- Increases flexibility of sharing
  - Share either single page or entire segment. How?

Disadvantages of Paging with Segmentation

Potentially large page tables (for each segment)

- Must allocate each page table contiguously
- More problematic with more address bits
- Page table size?
  - Assume 2 bits for segment, 18 bits for page number, 12 bits for offset

Each page table is:
= Number of entries * size of each entry
= Number of pages * 4 bytes
= 2^18 * 4 bytes = 2^20 bytes = 1 MB

- Inherit other problems of segmentation:
  - Reserving bits of virtual addr prevents using some addrs
  - Need compiler and loader support to generate appropriate virt addrs

Other Approaches

1. Inverted Pagetables
2. Segmented Pagetables
3. Multi-level Pagetables

"Paging the page tables"